一篇搞懂 AI Coding Agent 的 Token 成本控制

腾讯技术工程 Jun 15, 2026, 2:27 AM 广东 160 listened



作者：devinyzeng

> 适用：CodeBuddy、Cursor、Codex、Gemini CLI 等这类 AI Coding Agent。
>
> 读完你会得到三样东西：一个正确的心智模型、一份今天就能做的行动清单、几套继续往下压成本的工程方法。

月初看账单，吓一跳。

也没问多少问题，Token 怎么烧成这样？

后来认真拆了一轮，才发现一个反直觉的事实：

你问的那句话，在每次请求里可能连 1% 都不到。

真正的成本大头藏在别的地方。这篇文章，就是把这个“别的地方”讲清楚，然后给一套从日常使用习惯、模型路由，到工具层、代码图谱、多 Agent 协作的完整优化方法。

第一章 账单都花在哪了

很多人一上来先优化提问方式：把句子写短一点，把形容词删掉一点，把问题问得更“像 Prompt Engineer 一点”。

方向不算错，但经常抓错重点。

因为在 AI Coding Agent 里，真正决定账单的，通常不是你手里敲进去那几十个字，而是系统为了让模型“看起来懂上下文”，自动帮你带上的那一大坨东西。

先把一轮请求拆开看清楚，再谈优化。否则你会一直在省零钱，漏掉大头。

1.1 你问的那句话，其实最便宜

先看一个典型的请求结构：

```
System Prompt          5K
项目说明文档           10K
Skill 定义             20K
Tool / MCP 定义        30K
历史会话              100K
代码文件               50K
用户问题               0.1K
```

这不是所有产品都一样的精确账单，而是一种很常见的分布。

它传递的信息很明确：

> **贵的是系统塞进去的东西，不是你写的那句话。**

很多人本能地把优化方向定为“怎么把问题写短点”。这当然不是坏事，但它抓不到主要矛盾。因为模型真正收到的，不是“你的问题”，而是“带着整包上下文的问题”。

可以把一轮请求近似理解成这个公式：

```
总成本 ≈ 固定前缀 + 会话历史 + 运行时检索 + 工具往返 + 模型输出
```

这里面有三层东西：

- **固定前缀**：System Prompt、Skill 定义、Tool / MCP 定义、稳定背景文档
- **半固定上下文**：项目说明、Repo Map、Memory、长期约束
- **动态上下文**：聊天历史、代码片段、检索结果、工具返回、你这轮新问题

你那句提问，往往只是最后一撮“点火器”。它负责触发任务，但通常不是成本主体。

这个结构可以画成一张很直观的图：



这也是为什么很多人会产生错觉：

```
我明明只问了一句话，怎么这么贵？
```

答案是：**你只问了一句话，但系统替你背了一整车背景。**

当你理解这一点，后面很多优化动作就会突然变得合理：为什么要开新会话，为什么要压缩历史，为什么要少装 Skill，为什么要做代码图谱。这些动作看上去分散，本质都在做同一件

事：

减少重复上下文。

1.2 “它明明记得”只是一种幻觉

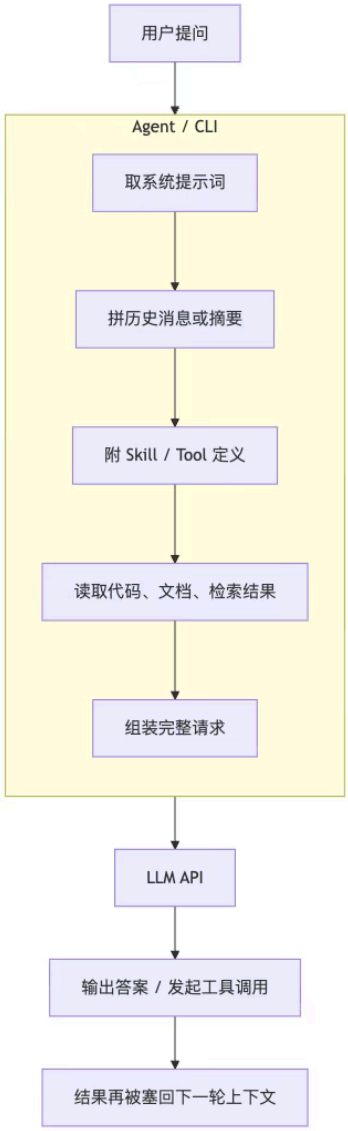
AI Coding Agent 用久了，很容易产生一种感觉：

```
它好像一直记得我们聊过什么
```

但实际上，大模型本身通常是无状态的。

真正“记得”的，不是模型，而是包在模型外面的 Agent / CLI / 平台层。它在每一轮请求前，把历史、规则、工具、代码、文档重新拼起来，再发给模型。所谓“记忆”，很多时候只是“再次传入”。

这个过程可以简化成：



这件事为什么重要？因为它直接决定了成本结构。

第一，**会话越长，后续每一轮越贵**。因为历史不是“存在那里等模型自己想起来”，而是很可能在每轮都要重新带过去。

第二，**工具越多，常驻定义越重**。一个 Tool / MCP 不只是“多一个按钮”，而是多一段要让模型理解的说明、参数模式和调用约束。

第三，**工具调用会形成回路**。模型先读一大包上下文，再调用工具；工具返回一段结果；结果又被塞回上下文，再进入下一轮。于是一次任务，不是一次计费，而是一连串“请求—返回—再请求”的链条。

所以 AI Agent 最怕的不是问题复杂，而是：

- 会话越滚越长
- 工具越堆越多
- 背景越塞越满
- 每次都从零找信息
- 输出不合格反复重试

很多人说“它明明记得”。更准确的说法其实是：

> **不是它记得，而是系统在一遍遍提醒它。**

1.3 五种成本，不止是“输入字数”

做成本优化，先别把 Token 理解成“字数”。

Agent 场景里至少有五类开销，要分开来看：

成本类型	它是什么	为什么会放大
输入 Token	系统提示词、历史消息、代码片段、检索结果、工具定义	每轮都可能重复携带，是最大头
输出 Token	模型最终回答的文字	回答越长、越啰嗦、越容易失控
推理 Token / thinking budget	模型内部用于思考和规划的预算	简单任务如果走高推理档位，会明显溢价
工具往返成本	工具描述、调用参数、返回结果进入上下文	一次工具调用，常常比原问题还长
重试成本	第一次做错后重新来一轮	每修一次，都可能把整包上下文再付一遍

这里最容易被低估的，往往是后两项。

工具往返成本的问题在于：你以为自己只是“让它查个文件”，但模型其实经历的是：理解工具定义 → 生成参数 → 接收返回值 → 再结合返回值继续思考。对人来说只是一个动作，对系统来说可能是多段上下文交换。

重试成本更隐蔽。真正烧钱的，常常不是第一次调用太长，而是第一次不合格——格式不对、结构错、找错文件、推理过度——于是又来一轮。

```
第一次没答对
→ 重新描述问题
→ 重新附带历史
→ 重新拉代码或工具结果
→ 再跑一轮模型
```

所以很多“看起来只补了一句话”的修正，实际含义是：

```
错误的第一次调用
+
第二、第三次修正调用
=
整包上下文被反复付款
```

这也是为什么“减少重试”本身就是一种非常硬的成本优化手段。格式约束更清楚、任务边界更明确、上下文给得更准，省下来的不只是一次输出，而是整条失败链路。

1.4 Prompt Cache：为什么它是所有优化的基础

理解了成本结构，就能理解为什么 Prompt Cache 这么重要。

很多人第一次听到缓存，会以为它缓存的是“答案”。其实更接近的理解是：**缓存的是稳定前缀的处理结果**。也就是——如果两次请求前半段几乎一样，服务端就不必每次都从头处理那一大段相同内容。[1]、[2]

原理可以先粗略理解成：

```
固定前缀 → 缓存命中 → 不用每次都从头处理
```

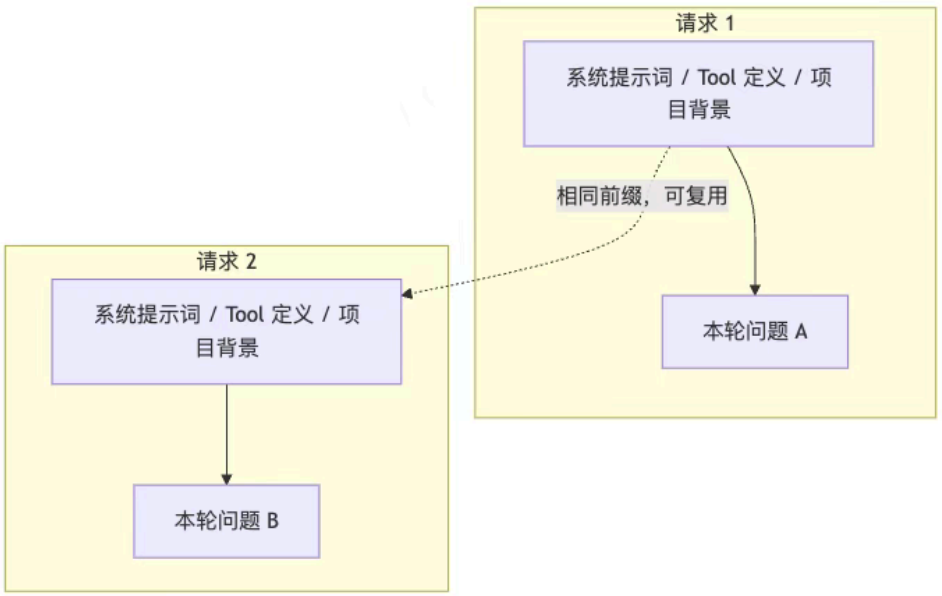
最容易被缓存的内容，通常是这些：

- System Prompt
- Tool / MCP 定义
- Skill 定义

- 长文档背景
- 稳定的 few-shot 前缀

为什么官方文档总在强调“静态内容放前面，动态内容放后面”？因为缓存命中的对象通常是**前缀**，不是整段任意位置的拼图。

看这张图就很容易懂：



这里有三个很关键的推论。

第一，Prompt Cache 省的不是首次成本，而是重复成本。 第一次发送长前缀时，通常还是要正常付费；价值在第二次、第三次、后续很多次复用时才会体现出来。

第二，缓存不是“写短”，而是“写稳”。 你天天改系统提示词、天天调 Skill Prompt，那缓存理论上存在，实践里也很难命中。

第三，缓存优化和上下文治理是一回事。 减少前缀抖动、把稳定内容前置、把变化内容后置，本质都在提升“可复用比例”。

所以 Prompt Cache 本质上不是让模型更聪明，而是让你别为同一段前缀反复买单。官方数据也说明了这一点：OpenAI 与 Anthropic 都提到，长前缀命中缓存后，输入成本和延迟都可能显著下降。[1]、[2]

> **Token 优化重点，不是把提示词写短，而是把前缀写稳。**

（缓存的具体配置方法，会在后文工具层详细展开。）

1.5 一张全图：五层模型

梳理完成本来源后，再看优化路径就会清楚很多。

这篇文章核心框架分五层：

层级	主题	作用	特点	位置
第一层	使用习惯	先减少无意义上下文	零成本，立刻可做	前半部分
第二层	模型路由	让不同任务匹配不同模型	ROI 最高	前半部分
第三层	Context 工程	稳定前缀，减少重复传输	系统级优化	后半部分
第四层	代码图谱	减少盲搜，缩短检索链路	收割检索成本	后半部分
第五层	Agent 架构	Orchestrator-Worker 编排，上下文隔离 + 并行执行	费用和时间双降	后半部分

它们不是并列的小技巧，而是一条从便宜到贵、从易到难的升级路径。

- **使用习惯**解决的是“无意义的历史和废 Token”
- **模型路由**解决的是“贵模型干便宜活”
- **Context 工程**解决的是“同样前缀重复发送”
- **代码图谱**解决的是“每次都从零找代码”
- **Agent 架构**解决的是“所有任务都塞给同一个大上下文”

这五层背后其实只有一个总目标：

- 让模型少看无关内容
- 让便宜模型多干标准活
- 让贵模型只做高价值推理

还有一条贯穿全文的线：**观测与预算治理**。

没有数据，优化就会变成体感。你觉得自己“省了”，但不知道省在哪，也不知道是不是用质量换的。这个话题会在后文继续展开。

到这里，第一章真正想建立的心智模型可以浓缩成一句话：

> **AI Coding Agent 的成本，本质上不是“你问了什么”，而是“系统为了回答你，重复搬运了多少上下文”。**

一旦你脑子里有了这张图，第二章那些“看起来很琐碎”的使用习惯，就不再是经验技巧，而会变成非常自然的工程动作。

第二章 使用习惯：最便宜也是最被低估的优化

如果你现在就想开始省 Token，最先动的不是架构，不是知识图谱，而是使用习惯。

这层几乎零工程投入，却经常拿到第一波最大收益。接下来的九个习惯，都是今天就能改的。

2.1 一个 Session，一件事

很多人把 AI Agent 当成永不关闭的长会话：上午修 Bug，下午写文档，晚上聊架构，第二天接着来。

体验很顺，成本是灾难。

原因很简单：Session 越长 → 继承的历史越多 → 每一轮越贵。

> **一个 Session 只服务一个目标。**

修 Bug 开一个，做重构开一个，写文章开一个，查线上问题开一个。别混着来。

Topic 切分就是最便宜的 Context 管理。

2.2 长会话不压缩，就是负债

很多人舍不得清历史，觉得“越完整越稳”。

模型不需要你的完整试错过程。它需要的是：

- 现在要干什么
- 做完了什么
- 哪条路不通
- 卡在哪了
- 下一步怎么做

`/compact` 做的，本质上是这一件事：

```
1 把完整历史
2 压成可继续工作的状态
```

对话记录不是资产，未压缩的长对话是负债。它让后续每一轮都背着越来越重的历史包袱。

2.3 聊天记录不是数据库

很多团队把背景、决策、约束、待办全挂在会话里，指望 Agent 一路记到底。

后果：会话越来越长，状态越来越难提炼，成本越来越高。

真正该长期保留的信息，外置出去：

- 项目文档
- Summary 文件
- Memory 文件
- Repo Map / Knowledge Graph
- 任务清单

让会话只承载“当前工作状态”，而不是全部项目历史。

2.4 少说废话，也是省 Token

很多人优化只盯输入，忘了输出一样贵。

编码场景最浪费的回答不是“错”，是“废”：先复述一遍问题，再讲一段众所周知的背景，然后一堆礼貌性铺垫，最后才给结论。

如果你只需要 diff、步骤、结论、表格、JSON——直接要求它这么给。

这类约束值钱，因为它同时省两类：

1. 输出更短
2. 结果更可用 → 重试更少

在很多日常任务里，一句指令就够了：

```
直接给结论，不要复述问题，必要时再展开
```

Caveman 这类思路受欢迎，不是“简陋”，而是直接打掉高频废 Token。

2.5 Skill ≠ 免费能力

Skill 有价值，但也背成本。

每个 Skill 都带着 Description、Instructions、Examples、触发逻辑。这些信息如果常驻，就会进上下文。

问题不是“装不装”，而是：

> **哪些常驻，哪些按需加载。**

高频、稳定、通用的 Skill 常驻。低频、长说明、低复用的按需触发。

和后端服务治理同理：常驻能力要尽量少而精。

2.6 MCP 越多，不一定越强

MCP 的最大诱惑是让 Agent “什么都能接”。

但从成本看，每多一个 MCP，就多一份 Tool Definition、多一层选择成本。

很多人装了 GitHub、Notion、Jira、Slack、Browser、Filesystem、Kubernetes、Docker 和各种内部系统。看起来很强，实际高频用的就两三个。

工具太多不只是“文档长”的问题：

- 选择空间变大 → 决策变慢
- 错误调用概率变高
- 每轮携带定义更重

所以工具治理和依赖治理一样：关键不是能不能装，是有没有必要常驻。

2.7 CLI 优先于 MCP

这是一个有点尖锐但很实用的原则：

```
1 能 CLI 解决 → 尽量 CLI
2 能脚本解决 → 尽量脚本
3 最后才考虑 MCP
```

很多操作已经有非常成熟的 CLI：`gh pr create`、`kubectl get pods`、`docker ps`、`git diff`。

AI 直接走命令更轻，不需要额外拉一整套工具说明。

对于国内常见的研发平台，也有专门为 AI Agent 优化的 CLI 工具可以直接用：

- **tapd-ai-cli**：腾讯 TAPD 的 AI Agent 专用 CLI，支持需求、缺陷、任务的查询和更新。直接用 `tapd story list`、`tapd bug create` 之类的命令操作 TAPD，比拉一套 MCP 轻得多，还能用 `tapd skill init` 一键生成 SKILL.md 配置。
- **gongfeng-cli**：腾讯工蜂代码托管的 AI Agent 专用 CLI，专门针对 Agent 的认知模式优化了输出格式（Markdown + JSON 混合），支持 PR 创建、代码评审、项目管理等完整工作流，Token 消耗显著低于同等 MCP 方案。

这两个工具的共同思路值得借鉴：**专为 Agent 设计，而不是给人看的 UI 套了个 CLI 壳**。输出格式、信息密度、命令发现机制，都针对 AI 的调用模式做了优化。

MCP 的价值在：跨系统编排、结构化返回、权限统一收敛、内部业务能力包装。

所以不是“CLI 永远优于 MCP”，而是：

> **已有成熟命令链路时，不要为了看起来高级而过度工具化。**

2.8 引用文件时带完整路径

这是一个极其容易忽视的小习惯，但省的 Token 实实在在。

很多人引用文件时习惯只写文件名：

```
1 看一下 config.go 的问题
2 帮我修改 utils.py
```

AI 收到这类请求，往往要先搜索整个项目找这个文件在哪，有时候还会搜到同名文件或路径不确定，再次搜索确认。这些搜索调用和返回结果，全部计入 Token。

更直接的做法：

```
1 看一下 src/config/config.go 的问题
2 帮我修改 @/Users/yourname/project/utils.py
```

用 `@` 符号加上相对路径或绝对路径，Agent 直接定位读取，不需要搜索任何中间步骤。

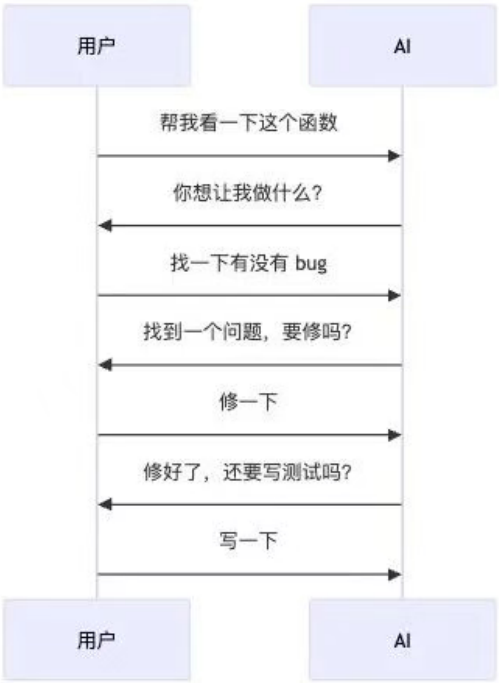
这个习惯对大型项目更显著——项目越大，搜索代价越高，目录层级越深，搜索越容易绕弯。

```
1 只写文件名 → 搜索 → 可能多次确认 → 读取
2 完整路径   → 直接读取
```

一个 `@路径` 节省的，是一整条搜索链路。

2.9 把意图一次说完，别聊天式拆碎

AI Agent 的使用场景里，有一种很常见的低效模式：



表面上看，这是“正常沟通”。实际上，每一轮来回都在消耗 Token：重新组装上下文、重新带入历史、重新确认状态。四轮对话的成本，往往是一次完整表达的好几倍。

更省的做法是一次把意图说清楚：

```
1 看 @src/order/service.go 的 CreateOrder 函数，
2 找出潜在的 bug，修复，并为修复后的函数写单测。
```

这不是要求你写得长，而是要求你**把目标说完整**。一次对话，Agent 一次性跑完整个任务，省去了所有中间确认的来回成本。

越具体、越完整的指令，浪费的轮次越少。这和程序员提 Issue 的逻辑是一样的：把背景、期望、验收条件都写清楚，沟通成本才不会比写代码还贵。

第三章 模型路由：别让贵模型干便宜活

习惯层做好之后，下一步收益最高的，是模型路由。

很多人一上来就犯一个错：所有环节默认最强模型。看起来稳，实际上最烧钱。

3.1 匹配优先，不是便宜优先

模型路由不是“一律用便宜的”，而是先把任务类型分对：

- 复杂任务 → 强模型
- 简单任务 → 便宜模型
- 重复任务 → 稳定模型

架构设计需要长链路推理，该用贵的就用。写单测、补注释、生成提交信息的，便宜模型完全够用。大规模批量分类和摘要，适合低单价模型或离线批处理。

3.2 一张表说清楚

任务	推荐模型档位	理由
写 UT	便宜模型	模式稳定、模板化
写 commit	便宜模型	输出短、风险低
Code Review	中高档模型	需要语义理解
架构设计	强模型	长链路推理
复杂 Bug 分析	强模型	多步假设和验证
批量分类/摘要	低价或 Batch	规模大、成本敏感

选模型不是凭喜好，是按任务需要的能力。

3.3 先过便宜模型，再升级

很多任务不需要一上来就最贵模型。

更省的模式可以简化成一条升级链路：

```
便宜模型先跑
↓
判断复杂度
↓
需要才升级
```

例如：先让便宜模型分类（重构？Bug？文档？），先做初筛（PR 有没有高风险？），先做摘要（20 页 → 1 页），再交给强模型做推理。

这种级联工作流，既省钱，又让贵模型花在刀刃上。

3.4 不只换模型，也要调预算旋钮

现在很多模型提供成本旋钮：

- `reasoning effort`
- `thinking budget`
- `verbosity`
- `max output tokens`

它们影响的，其实是同一组东西：

```
想多深、答多长、花多少钱
```

Google Gemini 的 `thinkingBudget` 允许按任务控制思考 Token，简单任务甚至可以压到 0（关闭思考）。OpenAI 也建议很多工作负载从 `low` 的 reasoning effort 起步。

> **不是所有任务都要深度思考，也不是所有任务都要长答案。** 让一个模型用高推理档位做分类、摘要、提交说明，本质上是在拿推土机扫地。

3.5 Skill / Agent / Command 都应该绑定模型

模型路由不该只存在于“主对话框”的心智里，要落实到执行单元。

Skill 绑定模型：写 UT、commit、格式修复、样板代码，明确绑定便宜模型。

以 CodeBuddy 为例，SKILL.md 文件头部可以直接声明模型和上下文策略：

```
---
context: fork
model: deepseek-v4-pro
---
```

`model` 字段指定该 Skill 使用的模型，`context: fork` 表示把这类固定流程放到独立执行上下文里，适合任务边界清晰、可单独完成的 Skill。这样一来，低复杂度的 Skill 就能自动走便宜模型，也不必把主会话的全部历史都带进去。

斜杠命令（Slash Command）和 Agent 同样支持 `model` 参数绑定，在定义时就固化好模型选择。这样团队里每个人用 `/commit`、`/gen-ut` 这类命令时，都默认走便宜模型，不需要依赖个人记得切换。

Agent 绑定模型：

```
Planner  → 中高档
Coder    → 中档 / 便宜
Reviewer → 强
```

Command 绑定模型：很多命令本身就是固定模板，适合预设便宜模型。这类设计一旦固化，成本治理就从“靠人记得切”变成“系统默认更省”，前三章解决的是两件事：先减少无意义上下文，再别让贵模型干便宜活。

下面开始进入工具层和架构层：

- 先把高频进出上下文的内容压下来
- 再让 Agent 少走盲搜链路
- 最后把多 Agent 的职责边界拆清楚

第四章 上下文压缩：先把高频上下文压下来

如果说前面三章解决的是“少带无关上下文”，这一层解决的就是另一件事：**把那些一定会进上下文的内容，尽量压短。**

这四个工具方向不同，但可以叠加。你不一定要一次装全，但只要装上一个，通常就能立刻看到收益。

4.1 RTK：终端输出才是真正的 Token 大杀手

很多人把 Token 浪费归因于“聊太多轮”，但忽略了一个隐性大户：**终端命令的冗余输出**。

CodeBuddy 执行`cargo test`，成千上万行测试日志、警告、进度条、ANSI 颜色码全部涌进上下文。

实测：30 分钟开发会话对比（中等规模 TypeScript/Rust 项目估算）：

场景	Token 消耗
不用 RTK	~118,000
用 RTK	~23,900
节省	**80%**

按命令类别的典型压缩率：

命令类别	原始 tokens	RTK 后	节省
cargo test / npm test	25,000	2,500	**-90%**
pytest	8,000	800	**-90%**
vitest run	102,199 字符	377 字符	**-99.6%**
ls / tree	2,000	400	-80%
grep / rg	16,000	3,200	-80%
git status	3,000	600	-80%
git diff	10,000	2,500	-75%

命令类别	原始 tokens	RTK 后	节省
cat / read	40,000	12,000	-70%

> **说明**：这张表混合了 token 和字符两种统计口径。前几行是 token，`vitest run` 这一行是字符长度，用来说明压缩幅度，不能和 token 行直接横向比较。

RTK 的四层过滤策略

1. **智能过滤**：剔除注释、空行、ANSI 颜色码、进度条、无关警告
2. **分组聚合**：同类输出合并展示（搜索结果按文件分组、错误按类型归类）
3. **智能截断**：按信息密度取样，保留关键片段，砍掉重复长尾
4. **去重合并**：「连接超时」重复 10 次 → 「连接超时（×10）」

5 分钟安装

```
# macOS（推荐）
brew install rtk

# Linux / WSL
curl -fsSL https://raw.githubusercontent.com/rtk-ai/rtk/master/install.sh | sh
```

注意：安装后用 `rtk gain` 验证。如果命令不存在，可能装了同名的 Rust Type Kit，需卸载重装。

```
# 全局启用 CodeBuddy（推荐）
rtk init -g --agent codebuddy

# 仅启用 Hook，不修改 CODEBUDDY.md
rtk init -g --hook-only
```

重启 CodeBuddy 后自动生效，所有命令透明经过 RTK 过滤，开发者无感知。

```
# 常用命令
rtk gain                # 查看节省统计
rtk gain --history      # 带命令历史
rtk discover            # 扫描哪些命令还没用 RTK

# Git（压缩率 60-80%）
rtk git status
rtk git diff
```

```
# 测试（压缩率 90-99.6%）
rtk test cargo test
rtk vitest run
```

4.2 Caveman：压缩输出端的废话

RTK 压缩的是命令输出（输入 Token），Caveman 压缩的是 AI 回复（输出 Token）。两者方向不同，可以叠加。

核心数据（来自 arXiv:2604.00025）：

任务类型	正常输出	Caveman	节省
解释 React bug	118 tokens	15 tokens	**87%**
配置 PostgreSQL	234 tokens	38 tokens	**84%**
Docker 多阶段构建	104 tokens	29 tokens	**72%**
Code Review PR	67 tokens	39 tokens	41%
平均	—	—	**65-75%**

Caveman 有四种模式，按压缩程度递增：

- `/caveman lite`：删填充词，保留冠词，专业风格（适合正式输出）
- `/caveman`（full 默认）：删冠词，碎片句，平衡压缩
- `/caveman ultra`：极度压缩，用 `A→B→C` 箭头因果链
- `/caveman wenyan`：文言文模式（理论上 token 效率最高的书面语）

实际效果示例（full 模式）：

```
问：为什么 React 组件不断重渲染？

普通输出（69 tokens）：
"The reason your React component is re-rendering is likely because you're
creating a new object reference on each render cycle..."

Caveman 输出（19 tokens）：
"New object ref each render. Inline object prop = new ref = re-render.
Wrap in useMemo."
```

安装

```
git clone https://github.com/studyzy/caveman
cd caveman && ./install.sh
# 选中 [2] CodeBuddy Code 确定即可
```

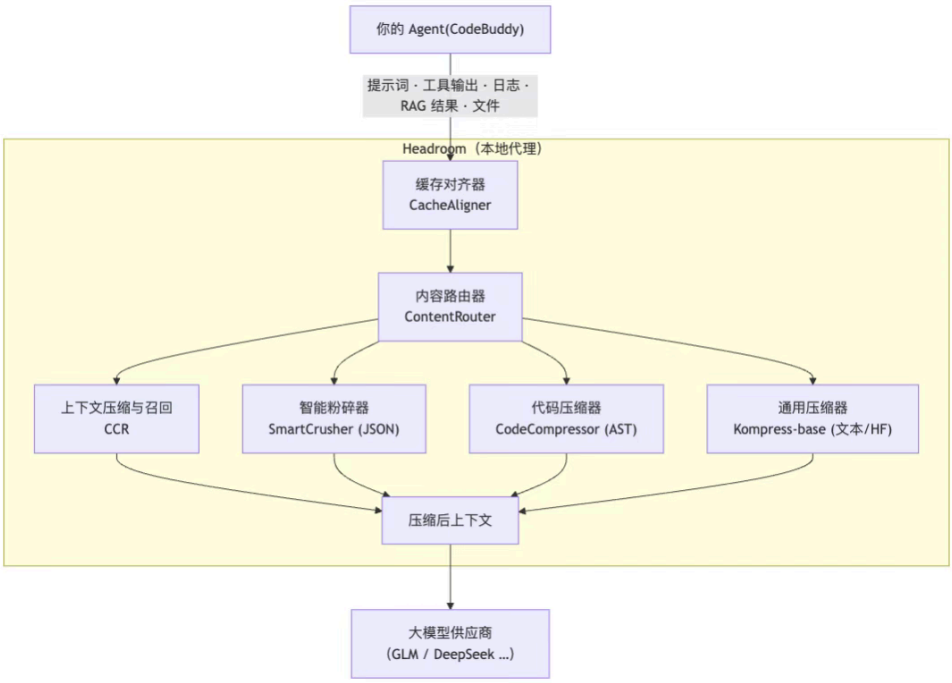
> **说明**：Caveman 官方目前还没有支持CodeBuddy所以使用 **studyzy Fork 版**（已增加 CodeBuddy 支持）。

安装后在 CodeBuddy 里输入 `/caveman` 即可激活。模式在会话内持续生效，输入 `stop caveman` 关闭。

4.3 headroom：压“进上下文的所有内容”

RTK 压的是命令输出，Caveman 压的是 AI 回复。headroom 压的是第三个方向：**所有读进上下文的内容**——文件内容、工具调用返回值、会话历史。

先看一张图，理解 headroom 在整个链路中的位置和它内部做了什么：



这张图解释了三件事：

1. **headroom 首先是代理层，不只是压缩器**——Agent 不直接连 LLM，而是先经过本地的 headroom 代理入口。提示词、工具输出、日志、RAG 结果、文件等上下文，都会先在这里被接住、处理、再转发。
2. **代理层内部是一条处理流水线**——先由 CacheAligner 稳定前缀，帮助 Prompt Cache 命中；再由 ContentRouter 判断内容类型，把 JSON、代码、文本分别交给 SmartCrusher、CodeCompressor、Kompress-base。
3. **CCR 负责“可逆压缩”**——原始数据全部保留在本地，模型只收到压缩版；需要细节时再通过 `headroom_retrieve` 按需取回。不是删掉信息，而是延后加载细节。

实测效果（真实 Agent 工作负载）：

场景	压缩前	压缩后	节省
代码搜索（100 条结果）	17,765	1,408	**92%**
SRE 故障排查	65,694	5,118	**92%**
GitHub Issue 分类	54,174	14,761	**73%**
代码库探索	78,502	41,254	**47%**

更重要的是，压缩后的准确率没有下降。在 GSM8K（数学推理）、TruthfulQA（事实准确性）等标准基准上，压缩前后的得分基本持平甚至略优。

安装

```
pip install "headroom-ai[all]"
```

> **说明**：如需在 CodeBuddy 中使用，可使用 **studyzy Fork 版**（已增加 CodeBuddy 支持）。

接入 CodeBuddy

```
# 基础接入
headroom wrap codebuddy

# 推荐：同时开启跨 session 记忆 + 代码图谱集成
headroom wrap codebuddy --memory --code-graph
```

之后正常用 CodeBuddy，headroom 在后台透明处理压缩，无感知。

也支持 MCP Server 模式

如果不想用 wrap 方式，可以走 MCP：

```
headroom mcp install
```

安装后在 CodeBuddy 里自动注册三个工具：`headroom_compress`、`headroom_retrieve`、`headroom_stats`。

与 RTK 的区别

	RTK	headroom
压缩对象	终端命令的输出	所有进上下文的内容
工作方式	过滤 + 截断	可逆压缩 + 按类型路由 + 按需还原
安装方式	brew / curl	pip + wrap 命令
典型节省	89%	47–92%

两者方向不同，Headroom 甚至还集成了 RTK，可以叠加使用。如果只装一个，RTK 解决最大的单点问题（终端输出）；如果追求更全面的压缩覆盖，再加上 headroom。

4.4 context-mode：工具输出不再撑爆上下文

context-mode 解决的是一个具体痛点：**MCP 工具调用的返回值太大**。

一次 Playwright 快照 56 KB，二十条 GitHub issue 59 KB，一份访问日志 45 KB。用 30 分钟，40% 上下文就没了。然后 /compact 一压缩，CodeBuddy 忘了在改哪些文件、任务进行到哪了。

context-mode 是一个 MCP server 插件，四层能力一起上：

能力一：Sandbox 工具输出（98% 压缩）工具调用结果先沙箱化，不直接进上下文。315 KB → 5.4 KB。LLM 按需取回原始数据，不是真的删掉。

能力二：跨 compact 会话连续性所有文件编辑、git 操作、任务进度、错误信息全部记录到本地 SQLite。/compact 之后不丢失现场——context-mode 用 FTS5 全文索引按相关性取回，而不是把所有历史重新塞回去。

能力三：用代码代替读文件ctx_execute() 工具让 CodeBuddy 写脚本处理数据，而不是读 50 个文件进上下文：

```
// Before: 47 次 Read() = 700 KB
// After: 1 次 ctx_execute() = 3.6 KB
ctx_execute("javascript", `
  const files = fs.readdirSync('src').filter(f => f.endsWith('.ts'));
  files.forEach(f => {
    const lines = fs.readFileSync('src/' + f, 'utf8').split('\\n').length;
    console.log(f + ': ' + lines + ' lines');
  });
`);
```

CodeBuddy 生成并运行脚本，只把结果（几行文字）放进上下文，而不是把 47 个文件的内容全部读进来。

能力四：不干预输出格式context-mode 只管数据往哪走，不管 CodeBuddy 怎么回答。如果你同时装了 Caveman，两者各管各的，不冲突。

安装（CodeBuddy 插件市场直装）

```
npm install -g context-mode
```

> **说明**：如需在 CodeBuddy 中使用，可使用 **studyzy Fork 版**（已增加 CodeBuddy 支持）。

重启 CodeBuddy 后生效。

验证

```
/context-mode:ctx-doctor
```

所有项显示 `[x]` 即正常。

常用命令

```
/context-mode:ctx-stats    # 查看节省统计，按工具分类
/context-mode:ctx-insight  # 打开本地分析看板（90 个指标）
```

四个压缩工具一览

工具互补，不互斥。从左到右压缩复杂度递增：

工具	压缩什么	典型节省	装法
RTK	终端命令输出 → 上下文	89%	`brew install rtk` + `rtk init -g`（CodeBuddy 用 Fork 版）
Caveman	AI 回复输出	65–75%	`install.sh`（CodeBuddy 用 Fork 版）
headroom	所有进上下文的内容	47–92%	`pip install headroom-ai[all]`（CodeBuddy 用 Fork 版）
context-mode	MCP 工具结果 + 会话连续性	98%（工具输出）	`npm install -g context-mode`（CodeBuddy 用 Fork 版）

最低成本起点：先装 RTK + Caveman，五分钟，立即见效。有时间再评估 headroom 和 context-mode。

第五章 代码图谱：让 AI 不再盲搜

前一层压的是“已经进来的东西”。这一层解决的是另一类浪费：**本来就不该读进来的东西。**

5.1 真正贵的是“找代码”，不是“读代码”

上下文窗口够大，不代表不需要图谱。

项目越大，AI 越容易陷入一个循环：

```
grep 一圈找入口
→ 读几个文件找关系
→ 发现漏了一个
→ 再 grep 一圈
→ 反复几轮才定位到
```

这个过程消耗大量 Tool Call 和上下文，而且结果还不一定准确。

代码图谱解决的是：**让 AI 在动手读文件之前，就知道该读哪里。**

5.2 Graphify：把代码库变成可查询的知识地图

Graphify 在 2026 年 4 月发布，短时间内 GitHub stars 超过 22k。

核心机制：用 Tree-sitter 解析代码构建知识图谱，AI 查图而不是反复读文件。

官方数据：**比直接读文件减少 71.5 倍 Token 消耗**。

安装和使用

```
# 安装
uv tool install graphifyy
# 或
pipx install graphifyy

# 注册到 CodeBuddy
graphify install --platform codebuddy
# 或
graphify codebuddy install
```

在 CodeBuddy 里执行 `/graphify`，自动扫描项目并生成三份产物：

```
> /graphify
```

```
Building knowledge graph...
  Parsed 247 files (TS/Prisma)
  Found 1,832 symbols, 4,217 edges
  Detected 14 communities (clusters)
  Generated:
    - graphify-out/graph.html          ← 交互式可视化
    - graphify-out/GRAPH_REPORT.md     ← 自然语言报告
    - graphify-out/graph.json          ← AI 查询用的数据
```

后续问跨文件问题时，AI 读 `graphify-out/graph.json`，而不是反复 grep 源码：

```
> 我想在 /api/v1/comments 加一个 PATCH 接口。
> 列出会受到影响的所有中间件、Prisma model、和测试文件。

影响分析（基于知识图谱）：

中间件链：
  - middleware/auth.ts → optionalAuth → requireAuth
  - middleware/rate-limit.ts（commenting 路径 30/min 限制）

Prisma model：
  - Comment（直接修改）
  - User（通过 authorId 关联）
  - Mention（通过 commentId 关联，PATCH 后需要重算）

测试：
  - tests/comments.test.ts
  - tests/integration/registration.test.ts（间接依赖）
```

不用图谱时，同样问题经常要靠多轮 grep 和补读文件来兜底，遗漏关键路径并不少见。用图谱后，定位和影响分析会稳定得多，尤其适合跨文件问题。

支持范围

- **30+ 种语言**：Python、TypeScript、Go、Rust、Java、C/C++、Ruby、C#、Kotlin 等
- **多模态**：SQL schema、Markdown 文档、PDF、图片（通过 LLM 提取语义）
- **增量更新**：git hook 触发，每次 commit 自动重建

与 CodeGraph 的区别

CodeGraph 是另一个方向的工具，基于 MCP Server + 持久化图数据库（Neo4j/KuzuDB），对 7 个真实开源仓库做了完整 benchmark（Claude Opus 4.8，2026-06-02 验证）：

平均数据：16% 更低成本、47% 更少 Token、58% 更少 Tool Call、22% 更快。

分仓库明细：

代码库	语言	成本	Tokens	工具调用
VS Code	TypeScript	-18%	-64%	-81%
Excalidraw	TypeScript	持平	-25%	-40%
Tokio	Rust	持平	-38%	-57%
Django	Python	-8%	-60%	-77%
Gin	Go	-19%	-23%	-44%
OkHttp	Java	-25%	-54%	-50%
Alamofire	Swift	-40%	-64%	-58%

注意：收益和项目规模、语言强相关。Rust/TypeScript 大型项目收益最显著，Java/Go 较小项目收益相对有限。

CodeGraph 安装：

```
# npm 安装
npm i -g @colbymchenry/codegraph

# 初始化项目
cd your-project
codegraph init -i
```

手动配置 CodeBuddy：

```
// ~/.codebuddy/.mcp.json
{
  "mcpServers": {
    "codegraph": {
      "type": "stdio",
      "command": "codegraph",
      "args": ["serve", "--mcp"]
    }
  }
}
```

核心工具：

- `codegraph_context`：找入口点和关键符号（第一跳）
- `codegraph_trace`：追调用路径（含动态派发）
- `codegraph_impact`：重构前影响分析

选哪个：个人项目、快速上手 → Graphify（Skill 形态，零配置）。团队、大型仓库、需要持久化查询 → CodeGraph（MCP 服务器，功能更完整）。

第六章 多 Agent 协作：边界清晰时，CodeBuddy 会更省 Token

前两层解决的是“单个上下文怎么更省”。这一层继续往前走：**别让所有任务都挤进同一个上下文。**

换句话说，前面讲的是“单个会话怎么省”，这里讲的是：**多个 Agent 怎么分工协作，才能在任务边界清晰时更省。**

CodeBuddy 本身就支持 subagent 调度、worktree 隔离、Skill 绑模型——不需要自己写 Orchestrator，用好内置能力就够了。

6.1 单 Agent 为什么越用越贵

复杂项目里，单 session 很容易演变成这样：

- 规划、写代码、跑测试、Code Review 全在一个会话
- 上下文越积越长（所有历史都往一个 session 里塞）
- 所有任务用同一个模型配置，便宜活也用最贵模型

职责越混，上下文越胖，成本越高。

6.2 subagent：任务隔离的最低成本方式

CodeBuddy 的 Skill 里可以直接调用 Agent tool，把子任务分发出去：

```
1 使用 Agent tool 分析这个 PR 的影响范围，
2 然后把结果返回给我，不需要读具体实现文件。
```

每个 subagent 都应该有独立的上下文边界——它只看到自己需要的内容，主 session 只看到结果。前提是子任务真的能拆开；如果每个 Agent 都要重复读同一批背景，拆得越多，反而越贵。

典型分工：

```
1 主 session（规划 + 决策）
2   ├── subagent A：影响分析（只看图谱，不读源码）
3   ├── subagent B：写实现（只看相关文件）
4   └── subagent C：跑测试 + 生成报告
```

与前文 3.5 的连接：subagent 可以单独绑定便宜模型。规划用强模型，执行用便宜模型，两者在不同 session 里，互不干扰。

6.3 Orchestrator-Worker 模式：把多 Agent 协作变成工程

前面几节讲的是"怎么给单个 Agent 减负"。这一节讲更进一步的思路：**用 Orchestrator-Worker 模式，把一个臃肿的长任务变成一个有分工的流水线。**

核心思路：

```
Orchestrator（协调器 Agent）
  ├── 负责规划、拆解、调度、汇总
  ├── 不亲自读大量文件，不亲自跑命令
  └── 只负责决策

Worker（子 Agent，按需派遣）
  ├── 每个 Worker 只做一件事
  ├── 只看自己需要的上下文
  └── 做完了把结果交回
```

为什么这个模式能省 Token?

一个单 Agent 处理复杂任务时，它必须同时承载规划、代码阅读、工具调用、生成、验证的所有上下文——哪怕它当前正在做的只是"改一个函数"。Orchestrator-Worker 模式的本质，是**让每个 Agent 只看和当前步骤相关的内容**，而不是把整个任务的所有背景都塞进每一次调用。

一个典型的成本对比：

```
单 Agent 全程跑：
  System Prompt(5K) + 历史(120K) + 规划(10K) + 代码(50K) + 工具结果(30K)
  = 215K tokens × N 轮

Orchestrator + Worker 分工：
  Orchestrator 轮次：System Prompt(5K) + 任务状态(2K) + 规划(3K) = 10K tokens
  Worker A 轮次：目标(1K) + 相关文件(8K) + 工具结果(5K) = 14K tokens
  Worker B 轮次：目标(1K) + 测试文件(6K) + 前步结果(3K) = 10K tokens

  同样完成任务，每轮成本压缩 5-10 倍
```

具体分工的一个真实例子——"修复 API 层 Bug + 补测试 + 写变更说明"：

```
Orchestrator（强模型）
  ├── 分析任务，生成工作计划到 .agent/plan.json
  ├── 派出 Worker A：定位 Bug（只给图谱和入口文件）
  ├── 派出 Worker B：修复代码（只给有 Bug 的那几个文件）
  ├── 派出 Worker C：补单测（只给修复后的文件 + 现有测试）
  └── 派出 Worker D：写 changelog（只给 git diff + 模板）

每个 Worker：
```

```
  - 独立上下文，完成即销毁
  - 绑定便宜模型（如 DeepSeek）
  - 并行执行（C 和 D 可以同时跑）
  - 结果写入共享文件，Orchestrator 汇总
```

在 CodeBuddy 里，这个模式可以通过 Agent tool 直接实现——Orchestrator 在主 Skill 或会话里运行，通过 `Agent tool` 分发子任务：

```
使用 Agent tool 执行以下子任务，上下文独立，不需要继承当前历史：

目标：定位 src/api/order.go 里的 CreateOrder Bug
上下文：只读 src/api/order.go 和 src/model/order.go
输出：把 Bug 描述和文件行号写入 .agent/findings.md
```

6.4 上下文隔离之后，数据怎么流转

这是 Orchestrator-Worker 模式最绕不开的问题：

> **上下文既然隔离了，Agent 之间怎么传递信息？**

答案：**通过共享外置文件，不通过会话历史。**

会话历史是每个 Agent 私有的，不同 Agent 的历史无法互通。但文件系统是共享的。这意味着上下文隔离和信息共享可以同时成立：

```
Agent A 完成工作
  → 把结果写入文件（.agent/step1_result.json）
  → 上下文销毁

Agent B 开始工作
  → 读取文件（.agent/step1_result.json）
  → 只看这个文件，不看 A 的历史
  → 把自己的结果写入 .agent/step2_result.json
```

这个模式有几个关键设计原则。

原则一：输出格式要结构化。 自然语言在 Agent 之间传递时容易产生歧义，也难以精确定位所需信息。结构化的 JSON 更紧凑、更可靠，下游 Agent 只需要读它关心的字段：

```
// .agent/findings.md → 改为 .agent/findings.json
{
"task": "locate-bug",
"status": "completed",
"findings": [
```

```
    {
      "file": "src/api/order.go",
      "line": 142,
      "issue": "并发场景下 inventory.Lock() 未释放",
      "severity": "high"
    }
  ],
"next_step": "fix-bug",
"context_needed": ["src/api/order.go:120-165", "src/model/inventory.go:30-55"]
}
```

下游 Agent 收到的指令里只需要包含：

```
读取 .agent/findings.json，针对其中的 findings 数组修复代码。
修复完成后将结果写入 .agent/fix_result.json，格式参考 .agent/findings.json。
```

原则二：用进度文件追踪状态。 复杂任务里，Orchestrator 需要知道每个步骤是否完成、是否失败、是否需要重试。这个状态本身也应该外置：

```
// .agent/progress.json
{
"task_id": "fix-order-bug-20260609",
"created_at": "2026-06-09T10:00:00Z",
"steps": [
    {
      "id": "step-1-locate",
      "status": "completed",
      "worker": "investigator",
      "output_file": ".agent/findings.json",
      "completed_at": "2026-06-09T10:02:30Z"
    },
    {
      "id": "step-2-fix",
      "status": "in_progress",
      "worker": "implementer",
      "started_at": "2026-06-09T10:02:35Z"
    },
    {
      "id": "step-3-test",
      "status": "pending",
      "depends_on": "step-2-fix"
    },
    {
      "id": "step-4-changelog",
      "status": "pending",
      "depends_on": "step-2-fix"
```

```
    }
  ]
}
```

Orchestrator 每次唤醒时，只需要读这一个文件就能知道任务进展。不需要回放任何 Agent 的历史会话。

原则三：每个 Worker 的 context 包精心裁剪。 新开一个 Worker 时，Orchestrator 应该明确告诉它"只读哪些东西"，而不是让 Worker 自己去探索：

```
# Orchestrator 派遣 Worker 时的指令模板

任务：为 CreateOrder 修复编写单元测试
上下文：
  - 修复内容：读 .agent/fix_result.json 中的 diff 字段
  - 现有测试风格：读 src/api/order_test.go（前50行）
  - 不需要读其他文件

输出：
  - 新增测试写入 src/api/order_test.go
  - 把测试覆盖情况写入 .agent/test_result.json

约束：
  - 只修改 order_test.go，不动其他文件
  - 测试不超过 80 行
```

这个指令本身很短（约 150 tokens），但它让 Worker 的上下文精准到最小必要集合。

原则四：临时文件及时清理。 `.agent/` 目录是临时工作区，任务完成后可以归档或删除：

```
# 任务完成后归档
mv .agent/ .agent-archive/fix-order-bug-20260609/

# 或直接清理
rm -rf .agent/
```

这样不会污染代码仓库，也不会把旧任务的上下文意外带入新任务。

6.5 并行执行：时间和成本同时压

Orchestrator-Worker 模式的另一个收益是**并行**。

独立的子任务可以同时启动，不需要等待。CodeBuddy 的 Agent tool 支持在一次 tool call 里发出多个并行指令：

```
同时启动以下两个独立任务（使用 Agent tool 并行调用）：

任务 A：为修复后的代码写单元测试
  - 读 .agent/fix_result.json
  - 读 src/api/order_test.go
  - 输出 .agent/test_result.json

任务 B：生成本次变更的 changelog 条目
  - 读 .agent/fix_result.json 中的 diff 字段
  - 参考 CHANGELOG.md 的格式
  - 输出 .agent/changelog_entry.md
```

两个 Worker 同时跑，各自只看自己需要的文件，没有上下文重叠。时间上并行，成本上独立，互不干扰。

实际加速效果取决于任务数量和独立性：

并行 Worker 数	实际加速倍数
2 个	1.5–1.8 倍
3 个	2.2–2.6 倍
4 个	2.8–3.4 倍

理论上 N 个并行就是 N 倍，实际略低，原因是 Orchestrator 本身有启动/汇总开销，以及 Worker 间有时有轻微的 I/O 竞争。但即便 50% 的并行效率，也意味着 4 个任务只花不到 2 个串行任务的时间。

什么样的任务适合并行：

```
✅ 适合并行：
  - 不同模块的 Bug 修复（改的文件没有交集）
  - 代码 + 测试 + 文档（三者可以同时生成）
  - 多个文件的格式化/重构（无依赖关系）
  - 影响分析 + 实现方案设计（可以同时推进）

❌ 不适合并行：
  - 有顺序依赖的步骤（先定位 Bug 才能修复）
  - 修改同一个文件的多个任务（会产生冲突）
  - 依赖上一步输出的任务（需要等待）
```

6.6 完整的编排流程：一个端到端示例

把上面三节串起来，看一个完整的端到端示例。

场景：给一个中型 Go 项目做 API 层重构——把分散的错误处理统一改成标准 Error Wrapper，同时补充缺失的单测，生成一份重构报告。

如果用单 Agent：

```
整个任务在一个 session 里跑
→ 随着轮次推进，历史越来越长
→ 到第 10 轮已经带着 150K+ 的历史
→ 每一轮都重新处理同一批背景
→ 中途如果 /compact，现场信息丢失
→ 总成本：约 800K–1.2M tokens
```

用 Orchestrator-Worker 编排：

```
阶段 0：初始化
  Orchestrator 读 graph.json，分析影响范围
  → 生成 .agent/plan.json（步骤、文件列表、依赖关系）
  → Orchestrator 本轮消耗：~8K tokens

阶段 1：分析（并行）
  Worker A：扫描每个 API 文件，找出非标准错误处理，结果写 .agent/audit.json
  Worker B：读现有测试，评估覆盖率缺口，结果写 .agent/test_gap.json
  → 两个 Worker 同时跑，各自消耗：~12K tokens
  → 总消耗（并行）：~12K tokens，耗时约 1 个 Worker 的时间

阶段 2：实现（可并行）
  Worker C：按 .agent/audit.json 逐文件替换 Error Wrapper（每次只处理一个文件）
  → 每次消耗：~6K tokens × 文件数

阶段 3：补测（串行，依赖阶段 2）
  Worker D：读修改后的文件 + .agent/test_gap.json，补测试
  → 消耗：~10K tokens

阶段 4：汇总（并行）
  Worker E：生成重构报告（读 .agent/audit.json + git diff）
  Worker F：生成 PR 描述（读 .agent/plan.json + .agent/ 各结果文件）
  → 两个 Worker 同时跑：~8K tokens

总消耗估算：约 100K–150K tokens（比单 Agent 节省 70-85%）
```

成本差异来源很直接：

1. **每个 Worker 只看当前步骤的相关内容**，不带全程历史
2. **便宜模型处理执行任务**，强模型只用于规划和决策
3. **并行减少时间成本**，等待时间从串行叠加变成并行中最长的那个
4. **进度文件替代会话历史**，Orchestrator 不需要回放所有轮次

到这里，这篇文章真正想建立的心智模型其实也可以浓缩成一句话：**不要只想着把单次回答压短，更要想办法让系统少把同一批背景反复搬进来。**

第七章 误区：很多"优化"其实在加成本

误区一：上下文越多越好。上下文越多不一定越准，但通常越贵。噪音越多，模型越容易分神。该读哪个文件，比读多少文件更重要。

误区二：MCP 越多越强。工具堆得越多，选择空间越大、决策越慢、错误调用概率越高。低频工具长期挂载，贡献的更多是成本。

误区三：所有 Agent 都上最强模型。这不叫优化质量，叫放弃路由。每个环节默认最贵模型，系统里就没有“资源分配”，只有“直接烧钱”。

误区四：聊天记录当长期记忆。长会话方便，但不是记忆系统。长期依赖会话承载全部背景，最终成本越来越高，关键信息越来越难提取。

误区五：只看单价不看总成本。便宜模型如果导致更多重试、更多搜索、更多上下文回填，总成本未必更低。真正该看的是**完成一次任务的总成本**。

误区六：Prompt 越短越好。压缩时丢掉了必要的 few-shot 或格式说明，导致第一次输出不合格，反复重试。压缩要精准，不是一味缩短。

第八章 结语

全文核心只有一句话：

```
AI Agent 成本优化
本质不是让你少问一句话
而是让系统少重复做无效工作
```

压成公式：

```
更低成本
=
更少重复上下文（RTK、Caveman、headroom、context-mode、会话管理）
+ 更合理模型路由（任务匹配、Skill 绑模型）
+ 更精准代码检索（Graphify、CodeGraph）
+ 更清晰 Agent 分工（subagent 隔离、worktree 并发、记忆外置）
```

行动清单

今天就能做

- `/clear` 切断无关历史，一个 Session 一件事
- 长会话及时 `/compact`，别让历史变包袱
- 指定输出格式，减少废话和重试
- 清点 Skill 和 MCP，低频的卸掉或按需加载
- 能 CLI 就先 CLI，别过度工具化
- 引用文件时用 @路径，不要只写文件名让 AI 去搜索
- 意图一次说完，避免聊天式拆碎指令

这周能做

- 按任务给模型分档：复杂任务用强模型，简单任务用便宜模型
- 给常用 Skill、Agent、命令绑定默认模型
- 安装 RTK，先压测试、git、搜索这类高频命令输出
- 需要更短回复时启用 Caveman，压掉输出端废 Token
- 评估 Graphify 或 CodeGraph，先让 Agent 知道该读哪里

这个月能做

- 把稳定前缀做成缓存友好结构，减少前缀抖动
- 评估 headroom 或 context-mode，控制更大规模的上下文输入
- 建代码图谱、Summary、Memory 等外置记忆层
- 把规划、实现、验证、Review 拆成边界清晰的多 Agent 工作流
- 用 Orchestrator-Worker 模式处理复杂任务：协调器负责规划，子 Agent 只看当前步骤所需上下文
- 建立 `.agent/` 目录约定：用结构化 JSON 文件传递 Agent 间数据，用 `progress.json`
- 追踪任务进度识别可并行的独立子任务，利用 Agent tool 并行调用压缩时间成本
- 给 Token、成本、重试率、工具调用次数建观测面板

参考资料

[1] OpenAI，《提示词缓存 | OpenAI API》

[2] Anthropic，《Token-saving updates on the Anthropic API》

[3] OpenAI，《使用 GPT-5.5 | OpenAI API》

[4] Google AI for Developers，《Thinking》

[5] Anthropic，《Prompt caching》

[6] RTK（Rust Token Killer）GitHub

[7] Caveman GitHub

[8] Caveman 论文：Brevity Constraints Reverse Performance Hierarchies

[9] Graphify GitHub

[10] CodeGraph GitHub

[11] headroom GitHub

[12] context-mode GitHub



阅读 1.3万

留言

写留言

还没有留言，来写首评